National College of Ireland

Project Submission Sheet

Student Name:	Meenakshi		
Student ID:	24235415		
Programme:	Msc in Cloud Computing	**Year:**	2025-2026
Module:	Fog and Edge Computing		
Lecturer:	Dr. Diego Lugones		
Submission Due Date:	17st April 2026		
Project Title:	Smart Chair Health Monitoring System: Fog and Cloud IoT Architecture on AWS		
Word Count:	2000		

Academic Integrity Certificate Verification Code: Q6lveHvE6b

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting **Monday, 19th January 2026, all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: 🔗 ttps://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature:	Meenakshi
Date:	17st April 2026

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.

3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date
Meenakshi	Msc in Cloud Computing	**17st** April 2026

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
[Insert Description of use]	
NA	NA

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text. Additional Evidence:

Academic Integrity Acknowledgement Supplement

The red square below indicates certificate number, please put this in the above field



NCI
Coláiste Náisiúnta hÉireann | National College of Ireland

This certificate is awarded to:

Meenakshi .

FOR COMPLETING THE PROGRAMME

Academic Integrity: For Students, Second Edition

January 18, 2026

Certificate Verification Code: Q6lveHvE6b

Course Code: MSCCLOUD

epigeum

Grok

Academic Integrity Subject Guide or how to access the site:
https://libguides.ncirl.ie/academicintegrity/epigeumforstudents
Completion of Epigeum Academic Integrity Online Course is Mandatory for all registered students of NCI

Smart Chair Health Monitoring System: Fog and Cloud IoT Architecture on AWS

Meenakshi
StudentId: 24235415
Fog and Edge Computing, MSc in Cloud Computing
National College of Ireland
Email: x24235415@student.ncirl.ie.

***Abstract*—This Project Focuses On Scable fog to cloud architecture of IoT to monitor human health condition in office environment. The Smart Chair Health Monitoring System gathers sensor information from a simulated chair that measures seat pressure, heart rate, posture angle, body temperature, and movement. Data gathered from these sensors is processed at the Fog node that runs on an EC2 instance and then sent to the cloud backend. The backend architecture consists of using the AWS API Gateway, Lambda function, and DynamoDB as storage services to process the sensor data. Also, the system includes a real-time dashboard that displays sensor readings through the use of HTML and Chart.js and triggers notifications in case of anomalies such as poor posture or lack of physical activity. This project shows how fog computing and cloud computing are applied for a scable IoT system. In this way, latency is minimized in addition to automatic scalability for real-time application. Keywords: Fog Computing, IoT, AWS, DynamoDB, Serverless Architecture, Health Monitoring**

I. Introduction

Employees in offices have to spend So much of hours sitting at their desks working on computers, which results in problems related to posture, spine, and physical exercise. Ergonomic monitoring systems can be used for detection of unhealthy behavior and alerting about the need to take actions. In terms of IoT solutions, sensors can be used to detect the current physical situation. At the same time, transmitting all collected data to the cloud can cause delay in data analysis and higher load on network resources. To solve this problem, fog computing is used. The goal of the Smart Chair Health Monitoring System project is the development of monitoring system based on fog-to-cloud approach. It will be developed for detecting and storing health data of office environment using virtual sensors that create ergonomic data. This project first gets started from terraform script to create the resources like synamoDB, S3, EC2 and uploads some necessary files in the S3.I have chose the terraform to provision the resource just to keep the track record of resources created and automate the process.After creation of resources I have manually uploaded fog and sensor file in EC2 and ran using python this invoke the lambda and the api link inside the processor and ingestion files inside lambda fetched the data and stores in dynamoDB table and it reflects on website when loaded. The following tasks need to be performed within this project: 1. Collect data from several simulated sensors. 2. Create fog node. 3. Create scalable backend based on AWS cloud services. 4. Store processed data in DynamoDB database. 5. Visualize sensor data in a dashboard.

II. Architecture and Design

The system follows a three-layer architecture consisting of sensors, fog nodes, and cloud services.

Component	Role	Technology
Sensors	Generates chair health data	Python simulator
Fog Node	Data processing and alert detection	AWS EC2 + Flask
API Gateway	Public cloud interface	AWS API Gateway
Data Processing	Event processing	AWS Lambda
Storage	Sensor data storage	DynamoDB
Dashboard	Data visualization	HTML + Chart.js
Terraform	Automates the process of resource creation	IaC tool

Fig. 1. Component and its role

System Data Flow Diagram 1. Sensor simulator produces health monitoring data.
2. This data is transmitted to the fog node through HTTP calls.
3. The fog node analyzes the data and sends alerts.
4. Analyzed data is then passed to the API Gateway.
5. Lambda get invoked and stores the data in DynamoDB.
6. The dashboard gets the data from the backend system and visualizes it and gives us the graph and live data for the chair which we are tracking.

III. Application Description

The Smart Chair system monitors the human body conditions using five simulated sensors.

Sensor types used

Key Features

Multi-Sensor Monitoring

The system collects data from multiple sensor types simultaneously.

Real-Time Alerts

Alerts are generated when abnormal conditions are detected and alerts are generated.

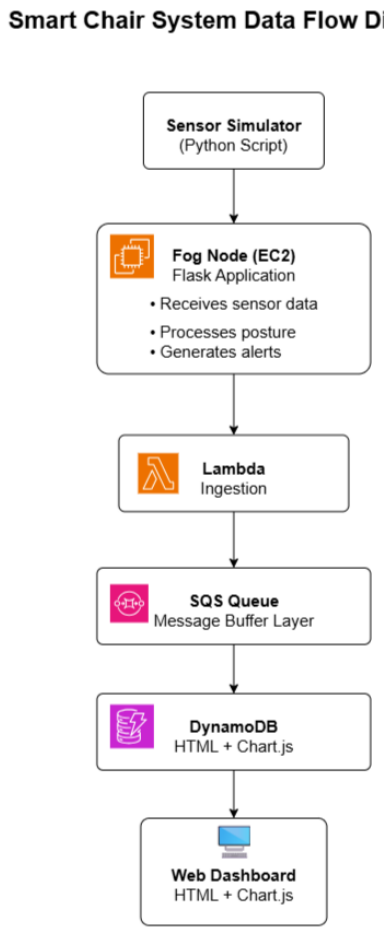


Fig. 2. Architecture Diagram

Example: posture angle ¿ 40° - Bad posture alert movement = false - inactivity alert

Real-Time Dashboard

The dashboard displays live sensor readings and graphical trends using Chart.js.

Design Justification Terraform

Its and IAC tool for provisioning the resources which has been used during the testing and overall experiment.

Fog Layer

The fog node implemented within this architecture is installed in an Amazon EC2 instance, acting the intermediate processing layer between the sensor simulator and the cloud infrastructure. Essentially, the fog node is the middleware that serves as a platform for processing sensor data by receiving, validating, and processing it locally before forwarding the data to the backend cloud environment. Deploying the fog node on the EC2 instance with the Flask web server enables the processing of the incoming data from the simulated sensor

Sensor	Description
Seat Pressure	Detects sitting pressure levels
Posture Angle	Measures back posture
Heart Rate	Monitors user heart rate
Temperature	Detects body temperature
Movement	Detects inactivity

Fig. 3. Table of Sensor Used

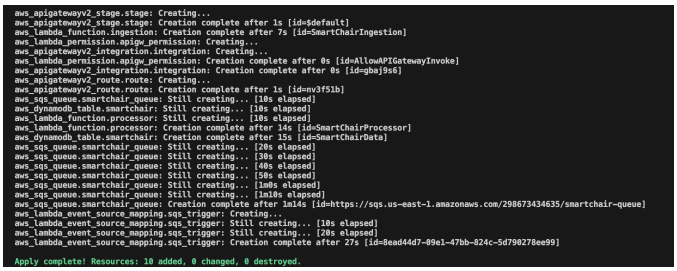


Fig. 4. terraform implementation

nodes and subsequent local computations.

In this project, within this architecture, the sensor simulator (sensorsimulator.py) generates the sensor data that needs to be forwarded to a the cloud. As a result, the data produced by the sensor simulator are forwarded to the fog node that performs certain data processing operations, including the filtering out incorrect data and separating the the readings. Since the data are processed locally, the fog node forwards only those readings that are required to perform further backend computations.

Finally, after the local processing of the incoming data, the fog node sends the data to the cloud by the invoking API Gateway endpoint, which will trigger AWS Lambda ingestion to operations.

Serverless Processing

AWS Lambda is used to process incoming events from API Gateway. Lambda automatically scales based on incoming traffic and eliminates the need for manual server management.

Cloud Storage

DynamoDB is chosen as a backend database because of its ability to scale rapidly along with time-series data and perform quick queries. Dashboard

Dashboard Visualization

Chart.js is utilized to provide visual representations of sensor data using JavaScript.

IV. Implementation

The system implementation consists of three main components.

Sensor Simulation

The sensor simulator generates random values representing

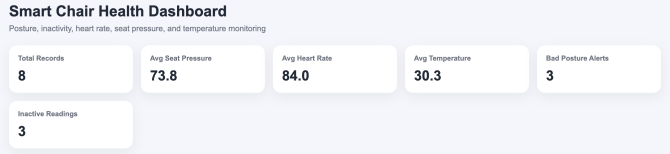


Fig. 5. Sensor Data

Human body posture conditions. Example payload:

”chair_id”: ”chair-01”,
”seat_pressure”: 75,
”posture_angle”: 50,
”heart_rate”: 85,
”temperature”: 32,
”movement”: false

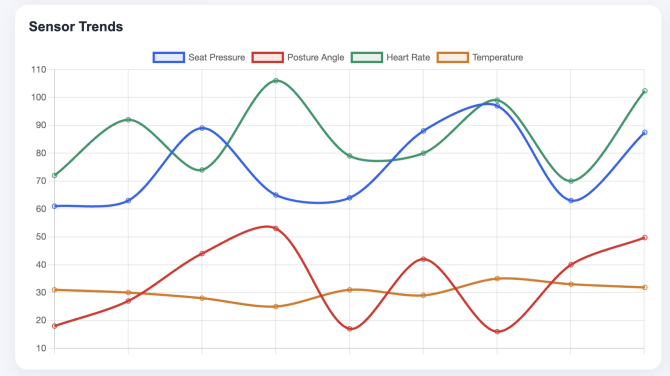


Fig. 6. Sensor Trends

Processing at the Fog Node Data processing on the fog node is done by using the Flask application that is hosted on an Amazon EC2 virtual machine. In this case, an intermediate processing layer is created between the sensors and the cloud backend that a facilitates the communication process. Flask refers to a light weight application development framework in the Python used for developing applications with REST APIs or HTTP requests.Therefore, when the data sent by the a sensor simulator reaches the fog node, the flask application starts processing an the data before passing it over to the cloud.

One of the tasks done by the fog node during data processing includes collecting and validating sensor data. The validation process involves checking the a format and validity an of the sensor data that is received. The validation checks include the presence of sensor id, proper formatting of the timestamp, the value ranges in the sensor readings. As the result, no invalid or corrupted data is passed over to the cloud backend.

Finally, after data collection and validation, the fog node processes the data and generates alerts where necessary.The processing and alert generation process involves checking the sensor readings and comparing them to a the set threshold values. Example for alert generation:

if posture angle greater than 40:
alert = ”Bad posture”

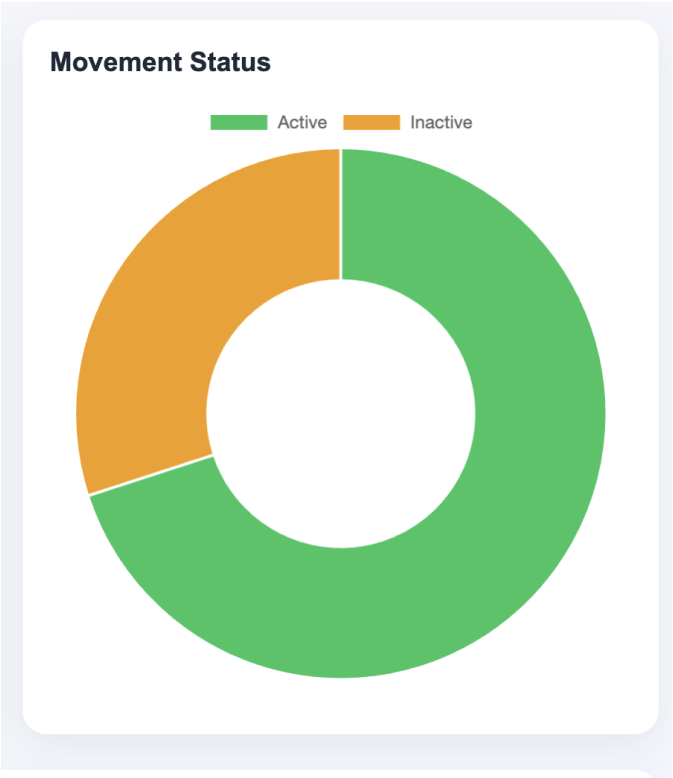


Fig. 7. Movement Status

Cloud Backend The backend architecture consists of API Gateway (smartchair-api), Lambda (SmartChairProcessor,SmartChairIngestion) and DynamoDB(SmartChairData).

Lambda functions receive data from API Gateway and store the processed sensor readings in DynamoDB.

Dashboard Implementation

The dashboard takes data from the backend API and displays:

- sensor trends
- alert notifications
- live fetched readings

Graphs are generated using Chart.js.

V. Results

The system successfully processes real-time sensor data and displays results through the dashboard.

Key outcomes include:

- Real-time data visualization
- Alert detection for poor posture
- Cloud-based data storage
- Scalable architecture

The system demonstrates effective integration between fog computing and cloud services.

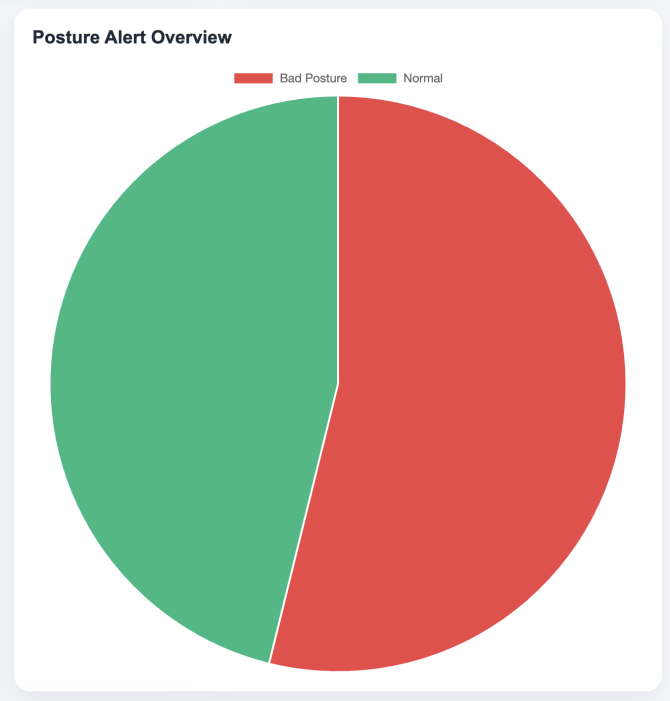


Fig. 8. Posture alert overview

Latest Sensor Records

Chair ID	Timestamp	Seat Pressure	Posture Angle	Heart Rate	Temperature	Movement	Alerts
chair-01	2026-04-15T23:45:01.285310	64	11	98	27	Active	No alerts
chair-01	2026-04-15T23:44:56.260979	80	20	81	27	Active	No alerts
chair-01	2026-04-15T23:44:51.228638	78	58	96	35	Active	Bad posture
chair-01	2026-04-15T23:44:46.199362	82	55	90	35	Active	Bad posture
chair-01	2026-04-15T23:44:41.184084	92	56	85	27	Active	Bad posture
chair-01	2026-04-15T23:44:36.159296	61	15	108	28	Active	No alerts
chair-01	2026-04-15T23:44:31.143980	88	50	103	32	Active	Bad posture
chair-01	2026-04-15T23:44:26.109920	63	40	70	33	Active	No alerts

Fig. 9. Latest Sensor Records

VI. Conclusion and Future Work

The above test is the working example of how to implement the use of a scalable IoT approach in monitoring the an human posture and health. In this context, sensor simulation, fog computing, cloudless computation, and real-time dashboard are all I achieved. By utilizing fog computing and a cloud services, we have created a highly efficient architecture without a any latency issues and maintaining the scalability.

One of the key a features implemented within the architecture under discussion is the usage of fog the computing.It means that some operations related to data processing are an not completed using solely cloud services but carried out a on-site,which means on the fog node, before the sensor data arrive in the cloud server. The local execution of some processes the significant speeds up data processing, reducing the volume of transmitted information and making the whole system much more efficient. Filter, validating, and the initial data analysis take place at the fog level, relaxes the burden of work for

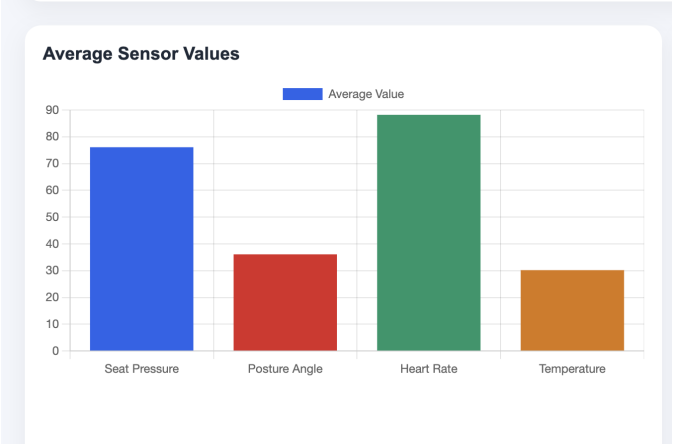


Fig. 10. Average Sensor Data

chair_id (String)	timestamp (String)	alerts	heart_rate	movement	posture_ang
chair-01	2026-04-16T14:45:17....	[{"S" : "Use...	106	false	37
chair-01	2026-04-16T14:45:23....	[{"S" : "Use...	95	false	30
chair-01	2026-04-16T14:45:28....	[{"S" : "Bad...	88	true	55
chair-01	2026-04-16T14:45:33....	[]	75	true	18
chair-01	2026-04-16T14:45:38....	[]	78	true	10
chair-01	2026-04-16T14:45:43....	[]	85	true	39
chair-01	2026-04-16T14:45:49....	[{"S" : "Bad...	74	true	49
chair-01	2026-04-16T14:45:54....	[{"S" : "Use...	102	false	40

Fig. 11. Dynamo Data

cloud servers.

the architecture under review involves the usage of both cloudless and cloud computations. Some computations can be performed locally, without the involvement of any remote cloud services.scalable and flexible storage and processing of data are executed using cloud computing.

The following could be some of the future improvements:
Implementation of hardware sensors
Notification mechanism for portable devices
Predictive machine learning based on posture
Multichair capability for larger office spaces

GitHub Link: https://github.com/Meenakshi-NCI/Fog-Edge_Project.git

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11]

REFERENCES

[1] A. W. Services, "Amazon api gateway documentation," AWS Documentation, 2026, [Online]. Available: https://docs.aws.amazon.com/apigateway/. [Accessed: Apr. 17, 2026].

[2] ——, "Amazon dynamodb documentation," AWS Documentation, 2026, [Online]. Available: https://docs.aws.amazon.com/dynamodb/. [Accessed: Apr. 17, 2026].

[3] ——, "Amazon ec2 documentation," AWS Documentation, 2026, [Online]. Available: https://docs.aws.amazon.com/ec2/. [Accessed: Apr. 17, 2026].

[4] ——, "What is aws lambda?" AWS Documentation, 2026, [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html. [Accessed: Apr. 17, 2026].

[5] ——, "Amazon simple queue service documentation," AWS Documentation, 2026, [Online]. Available: https://docs.aws.amazon.com/sqs/. [Accessed: Apr. 17, 2026].

[6] ——, "Boto3 documentation," AWS SDK for Python Documentation, 2026, [Online]. Available: https://docs.aws.amazon.com/boto3/latest/. [Accessed: Apr. 17, 2026].

[7] C. Contributors, "Chart.js documentation," Official Documentation, 2026, [Online]. Available: https://www.chartjs.org/docs/. [Accessed: Apr. 17, 2026].

[8] Cisco, "Fog computing and the internet of things," White Paper, 2016, [Online]. Available: https://www.cisco.com/c/en/us/solutions/trends/iot/fog-computing.html. [Accessed: Apr. 17, 2026].

[9] P. Project, "Flask web framework documentation," Official Documentation, 2026, [Online]. Available: https://flask.palletsprojects.com/. [Accessed: Apr. 17, 2026].

[10] K. Reitz, "Python requests: Http for humans," Documentation, 2026, [Online]. Available: https://requests.readthedocs.io/. [Accessed: Apr. 17, 2026].

[11] HashiCorp, "Terraform documentation," HashiCorp Documentation, 2026, [Online]. Available: https://developer.hashicorp.com/terraform/docs. [Accessed: Apr. 17, 2026].